

Introduction to Information Searching and Data Mining Practice 6 (and 7)

Department of Computing and Systems Engineering
School of Engineering and Architecture
University of Zaragoza

November 28th, 2025

1. Objectives

Information systems would be meaningless without their ability to easily retrieve registered information, process it, filter it, and obtain new information and knowledge. The mechanisms and tools for performing this processing and retrieval of information will depend primarily on the type of information we have, and the questions we ask of the information system.

If the information we have is documentary, i.e., natural language documents, we will need tools capable of analyzing texts, phrases, or words, capable of searching and retrieving specific digital documents based on different search criteria or selection, whether from the content of the documents themselves or their metadata.

On the other hand, if what we have are structured data, numerical, or categorical data related to any process or phenomenon, about which we want to perform analyses to understand and explain what happened, or predict what will happen, we will need to use data mining and machine learning tools on those data to obtain new knowledge.

In this practice, we will use two tools and execute some use cases as examples of what can be done by processing and recovering information. On one hand, we will become familiar with the **Apache Lucene** library. Apache Lucene allows incorporating indexing and information search mechanisms in applications implemented in various programming languages (*Java, .NET, Python, etc.*). In more detail, the objectives of this part are:

- Familiarize yourself with the Lucene library by testing the provided example. •

Implement a Java text-based application that enables:

- Indexing the contents of a specified directory.

- Adding a new content to the index.
- Searching documents based on keyword questions.

For the second part, we will use the *Weka* tool, which is an open-source tool with many implemented data mining and machine *learning* algorithms that are easy to use. The tool can read and analyze data from relational databases or different file formats. The objectives of this part are:

- Download and install the Weka tool.
- Become familiar with the Weka tool by testing the provided examples. •

Create a decision tree using the provided data.

2. Contents

The practice consists of two blocks. The first block involves getting familiar with the *Lucene* library, reviewing and testing the provided code to understand its basic functionality, and answering a series of questions related to the *Lucene* library. As a second task in this block, it is intended to develop a small text-based application that allows for the indexing and searching of documents in a specific directory containing *.txt* files.

2.1 Introduction to the Lucene Library

Lucene enables the creation of an inverted index, which adds documents for subsequent searching. The fundamental classes of the library are listed below:

- **IndexWriter:** Allows the construction and updating of the inverted index that will be used by the information retrieval system. *IndexWriter* is the "engine" of the *Lucene* system. In *Lucene*, indices are auto-incremental, meaning that once created, they can continue to add documents susceptible to being indexed.
- **Document:** Documents are the unit of indexing and searching in *Lucene*. An inverted index, in summary, is an index that tells you, for each search term (*word*), which documents contain that term. A *Document* object represents a single document. However, it should be noted that an object of this type does not necessarily correspond to a file, but rather constitutes the unit of indexing for the system's search engine, for example, it could represent a record in a database, a paragraph of text... It is the element pointed to by the inverted index. *Document* objects are modeled as a set of fields (*Fields*) in the form (name, value), and when indexing, we can decide which fields of the documents we want to index.
- **Field:** When creating a *Field* object, besides the field name and value, it must be indicated whether we want to store the value of the field in the index or not. If the field value is reasonably small, *Lucene* allows it to be stored in the index. *Fields* stored in the index can use the index instead of the original *Document* object to locate the data. In general, when we want to tokenize the field value, we use the *TextField* subclass, while if we don't want to tokenize, we use the *StringField*

subclass. Thanks to this division into fields, it is possible to work with documents that have a certain structure, for example, to index emails, you can consider four fields containing: sender, recipients, title or subject, and content; respectively, and perform searches based on that structure.

- **Analyzer:** *Analyzers* are responsible for processing texts to obtain their corresponding internal representations (*logical views*). They are used both in the creation of indices and in search processes, as queries must be processed with the same criteria as documents indexed. *Lucene* provides various analyzers, such as the *SimpleAnalyzer*, which simply divides the text into words and converts everything to lowercase, or the *StandardAnalyzer*, which also eliminates stopwords (by default, english language). With examples, it will be better understood the role of analyzers.
- **IndexSearcher:** It constitutes the basis for performing searches in a document index. To perform a search, you use the *search()* method, which provides documents that meet the conditions of the search indicated in an object *Query*.
- **Query:** It is an abstract class that provides the necessary mechanisms to formulate information needs.
- **Hit:** *Hit* objects represent each result obtained when performing some query on a specific index. Through these objects, you can access the document associated with the result. Additionally, each hit has an associated score (*score*) that represents the relevance of the associated document for the question asked.

2.1.1 Testing the Lucene Library

To familiarize yourself with the *Lucene* library, an example code is provided that uses the library. The example indexes four files containing environmental news articles (*uno.txt*, *dos.txt*, *tres.txt*, and *cuatro.txt*) and creates the index in a folder. Later, multiple searches are performed on the index, and the results are shown. The example code is encapsulated in an *Eclipse* project to facilitate its management from this *IDE*.

Perform the following queries and indicate what the search result is for each one:

1. contaminación
2. cambio climático
3. por
4. Aeropuerto

You can try making variations on the queries, such as:

- What happens if we use the "*StandardAnalyzer*" instead of the "*SimpleAnalyzer*"? What role does the "*stopwords.txt*" file play?
- What occurs if we put "*contaminacion*" or "cambio climatico" (without accents) in the search?
- What happens if we perform this search using the "*SpanishAnalyzer*"? Why does

this occur?

- What happens if we re-index all files each time we run the program, instead of simply reopening the previously created index?

2.1.2 Indexing and Searching Documents from a Specific Directory

Based on the example code, develop an application that performs searches on documents in a specific directory. A folder with a large number of EINA-related documents is included as part of the practice. The program should index this directory using the appropriate analyzer and allow searching for terms in that directory.

The application will offer an initial text menu:

1. Index a directory
2. Add a document to the index (optional)
3. Search term
4. Exit

The program will be tested against the provided set of documents, performing different questions based on terms considered relevant to you, noting the frequencies and relevance indices of those terms in the compiled "*corpus*".

2.2 Introduction to Data Mining with Weka

2.2.1 Installing the Weka Tool

First, you need to download the tool. You can use the following address:

<https://www.cs.waikato.ac.nz/~ml/weka>

Once downloaded, simply run the following command:

```
java -jar weka.jar
```

This will access the main screen of the tool, which defines four working environments: Simple CLI, Explorer, Experimenter, and KnowledgeFlow. Throughout this practice, you will use the visual Explorer environment.



The Explorer environment groups six modules: pre-processing, classification, clustering, association, attribute selection, and visualization.

2.2.2 A Classic: Iris Flower Classifier

In this first part, we will analyze a file that contains physical characteristics data for 150 flowers of the Iris species, indicating which subspecies each one belongs to (*setosa*, *versicolor*, *virginica*). The goal of this practice is to analyze the file and try to configure a classifier that, based on the data collected in the file, and knowing the characteristics of a new flower, tells us what subspecies it belongs to. In other words, we want the program, based on a set of examples, to be able to infer an algorithm for classifying flowers of the Iris species.

1. Click on the "*Explorer*" button to launch the *Weka* data explorer. This option allows you to load datasets and run classification algorithms. It also offers options such as data filtering, clustering, association rule extraction, and visualization.
2. Click on the "*Open file*" button to open the dataset and double-click on the "*data*" folder in *Weka*. *Weka* provides a set of small datasets for testing and practicing. One of them is the *iris.arff* dataset, which is what we will work with. Select the *iris.arff* file to load the dataset.

This dataset contains data about Iris flowers. It has 150 instances (*rows*) with 4 attributes each (*columns*) and one class attribute that indicates the type (*species*) of flower from Iris (*setosa*, *versicolor*, *virginica*).

3. Select and execute a data mining algorithm. Once the dataset is loaded, it's time to choose the data mining algorithm to model the problem and make analysis and/or predictions.

Click on the "*Classify*" tab. This tab contains the area where algorithms are executed against the dataset loaded in *Weka*. By default, the *ZeroR* algorithm is selected. To execute it, simply click on the "*Start*" button.

The *ZeroR* algorithm selects as the probable class for an instance the majority class in the dataset. Since all three Iris species are equally represented in the dataset, it chooses the first one: *setosa*, and uses that to make all predictions. This is a baseline for the dataset and measures how well all algorithms can be compared. The result is 33%, as expected (3 classes, each represented equally, assigning one of the three to each prediction results in a 33% classification accuracy).

Also observe that in the test options, cross-validation with 10 appears by default. This means that the dataset is divided into 10 parts: the first 9 parts are used to train the algorithm and the 10th part is used to evaluate the algorithm.

4. Select and execute another data mining algorithm. Click on the "*Choose*" button in the "*Classifier*" section and then click on "*Trees*" and select the *J48* algorithm. Finally, click on the "*Start*" button to run the new algorithm.

5. Analyze the results. After executing the *J48* algorithm, you can see the results in the "*Classifier output*" section. Like the previous algorithm, this algorithm was executed using *10-fold cross-validation*.

You can observe that the established model obtains the following result: it correctly classifies 144 of the 150 entries (96%), which is much better than the baseline (33%) from the previous algorithm. In the confusion matrix, you can see a table where real data is compared with predicted data. In the first row of the matrix, we see that there was an error and a setosa flower was classified/predicted as a versicolor flower, while in the last row, we see that 2 flowers of the virginica type were also classified as versicolor.

2.2.3 Association Rules

In this case, we will use a second dataset that collects meteorological data from various days along with an indication of whether it was possible or not to play tennis on that day. The goal is to be able to predict, based on the weather conditions of a specific day, whether it will be possible or not to play tennis.

1. Execution of classification algorithm with the dataset (*weather.nominal.arff*). This file contains data about the days when it was possible to play tennis based on various meteorological factors, and will be used to try to predict whether it will be possible to play tennis today or not.
 - 1.1. To work with the file, you need to load the data into the workspace by clicking on the "*Open file*" button in the Preprocess environment and selecting the corresponding file.
 - 1.2. When a document is loaded, information about its content is displayed. For example, it can be observed that this file contains 14 instances (*records*). Each instance consists of 5 values for the following attributes: outlook, temperature, humidity, windy, and play. If you select each attribute, in the right-hand part, you get information about its values in the dataset.
 - 1.3. You must select the "*Associate*" window.
 - 1.4. Then indicate which algorithm we will use to learn rules and parameters of this. In this case, we will consider *WEKA.associations.Apriori*. This algorithm can be configured with several options, for example, with the option *LowerBoundMinSupport* indicating the lower limit of support (or coverage) required to accept a rule; with the option *minMetric* indicating the minimum confidence to show an association rule; and with the option *numRules* setting the number of rules that will be shown.
 - 1.5. Click on the *Start* button. For each rule obtained, the coverage of the left-hand side and the rule, as well as the confidence of the rule, will be displayed. Examine each of the rules and analyze the results obtained. Write down the conclusions from this analysis, and explain in natural language what algorithm would be used to decide whether or not to play tennis based on the weather forecast for a given day.

2.2.4 Sentiment Analysis and Natural Language Processing

We will now apply data mining algorithms to text written in natural language (*text mining*). In this case, we will take data from film opinions obtained from the *IMDb* database, classified into two groups according to the global evaluation of the film by the spectator/opinion holder, whether it was positive or negative. The goal is to be able to decide, based on the text of an opinion, if that opinion was globally positive or negative.

1. Download the movie review dataset available at:
https://raw.githubusercontent.com/jbrownlee/Datasets/master/review_polarity.tar.gz
2. Through the *Weka* explorer, convert the dataset into a *.arff* file. For this, use "Open file" and select the directory that has two subfolders ("pos" and "neg"). You will get an error message because *Weka* does not know how to open the directory. Accept the error window and then open a dialog box where you can select the option "TextDirectoryLoader" by clicking on the button "Choose". Then, click on "OK". Finally, save the dataset (button "Save").
3. Convert the attribute "text" into a word vector. For this, in the "Filter" box, select (button "Choose") the operation "StringToWordVector" (*weka* $\square$ *unsupervised* $\square$ *attribute* $\square$ *StringToWordVector*).
 - a. Select the text attribute to apply this filter only to the first attribute (*text*) and not to the attribute *@@class@@*.
 - a. Click on "Apply" to execute the filter.
4. Apply a classification algorithm. For this, in the "Classify" tab, select (button "Choose") the algorithm "NaiveBayes". In the dropdown menu on the left side, select the attribute that you want to predict (in this case, "*@@classd@@*", which indicates whether the film opinion is positive or negative). Execute the algorithm (click on "Start"). Note the values obtained for the following results:
 1. Precision in detecting positive opinions.
 2. Precision in detecting negative opinions.
 3. Recall in detecting positive opinions.
 4. Recall in detecting negative opinions.
 5. Average precision.
 6. Average recall.
5. Now we will repeat the same operation, but preprocessing the input data slightly more: remove clearly irrelevant words from the word vector (numbers, signs like * and +, etc.); you can help yourself with patterns (button "Pattern"). Compare the global precision and recall obtained after removing these words: does it improve or worsen?).

6. Optionally, you can repeat the operation by selecting different parameters than the default when applying the conversion to a word vector. By clicking on the box next to the button "*Choose*", we can change the default parameters. Can you somehow improve the global precision and recall? (if so, indicate which parameters have more influence).

7. A document with answers to different points and with comments and explanations must be prepared.

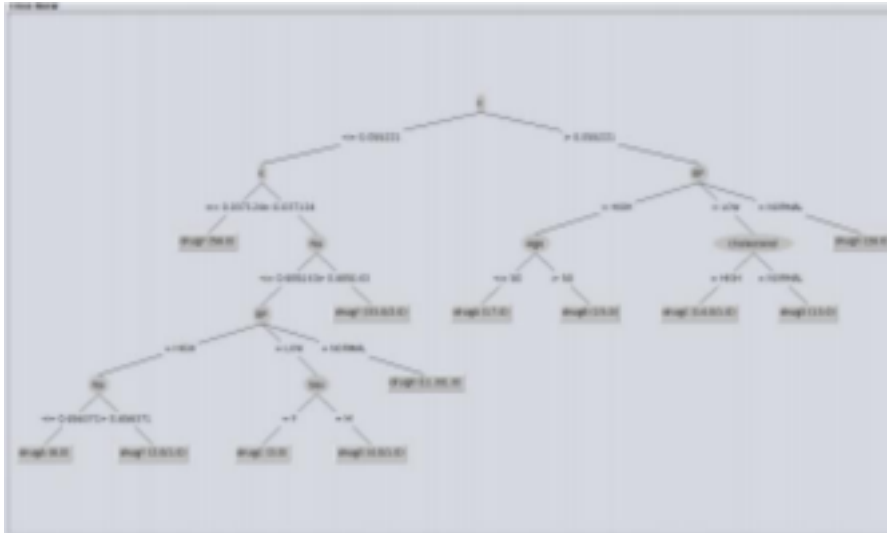
2.2.5 Delving Deeper into Classification Trees (Optional)

Finally, we will analyze some classification algorithms in a bit more detail. In this case, we are predicting the type of medication (drug) that should be administered to a patient with allergic rhinitis based on different parameters/variables. The variables recorded in each patient's clinical history are:

- Age: Age
- Sex: Gender
- BP (Blood Pressure): Blood pressure.
- Cholesterol: Cholesterol level.
- Na: Sodium level in blood.
- K: Potassium level in blood.

There are five possible medications: *DrugA*, *DrugB*, *DrugC*, *DrugX*, and *DrugY*. Data on the ideal medication for many patients has been recorded at four hospitals. We aim to determine the best medication to try for new patients. This would be the basis for a system that helps with pharmaceutical prescription.

1. From the Explorer, we load the file "*drugIn.arff*" provided in *Moodle*.
2. The first question we can ask ourselves is what medications are more common in general, to see if they all tend to be equally effective overall. To do this, we select the attribute *drug* and see the distribution by classes. It becomes evident that medication *Y* has success in more patients. This will be the medication that an algorithm like ZeroR recommends, with an accuracy of 45%. Using this algorithm as a recommender is equivalent to proposing medication *Y* (the most "effective") to the patient. If it doesn't work, we would give them *X* (the second most effective), and so on. This is a very common approach in medical practice, but we may be able to refine it a bit and get some new insights.
3. Using the *J48* classification algorithm (which we have seen before), with the dataset as our "training set", we find a decision tree that is quite more complex, with an accuracy of 97%.



If we try the same algorithm using *10-fold cross-validation*, the accuracy is slightly lower, at around 92.5%. This is normal. The accuracy is lower, but probably the system will perform better in real patient cases. It's likely that the rules obtained using the data as our "training set" are a bit over-optimized.

We could try other more complex and sophisticated algorithms, but we're going to do an analysis first, which may help us improve the predictive ability, even with the *J48* algorithm itself.

4. Sometimes, some of the variables we use in our analysis have a high correlation between them. In this case, the final result is distorted by this correlation, and it's better to use another auxiliary variable that simplifies the dataset and gives us a better precision. If we go to the "*Visualize*" tab in Explorer, we can see the relationship between pairs of variables. In color, we see the nominal variable we're trying to predict, which is the recommended medication. We'll spend some time looking at different graphs. Among them, we notice that there's a certain correlation between potassium level and age... but even better, between sodium and potassium levels. This graph allows us to almost immediately draw a straight line, below which *Y* is always recommended, and above which it's not recommended at all. This indicates an extremely high correlation between the two variables (*K* and *Na*), which we can simplify immediately.
5. We'll re-process the file, replacing the variables *K* and *Na* with their ratio, *K/Na*. To do this, we create a new derived attribute using preprocessing filters. From the "*Preprocess*" tab, we click on "*Choose*" in Filter and select the "*Unsupervised.Attribute.Addexpression*" filter. This filter allows us to add a new attribute based on previous attributes. To configure the newly created filter, we edit the text box and adjust the parameters:

Expression: a6/a5

Name: Na_to_K

a6/a5 means attribute 6/attribute 5, i.e., K / Na

We click "*Apply*" to create the new attribute. Once created, we can eliminate the attributes *Na* and *K* by selecting the checkbox and clicking "*Remove*".

6. We now have a retouched dataset. We go back to "*Classify*", and re-run the *J48* classifier. We'll see that we've improved the accuracy significantly, with a much simpler decision tree.

4. Practice Submission

The practice will be carried out in groups of three people. When the practice is completed, it must be submitted in a file called "*practica6_7_NIA1_NIA2_NIA3.tar*" or "*practica6_7_NIA1_NIA2_NIA3.zip*" (where *NIA1*, *NIA2*, and *NIA3* are the *NIA*s of the authors of the practice) with the following content:

1. A text file called "*autores.txt*" that contains the *NIA*, *surnames*, and *names* of the authors of the practice in the first lines of the file. For example:

NIA	Surnames	Name
545689	Rodríguez Quintela	Sabela
745689	Caamaño Cives	Antón
677893	López Marín	Adrián

2. A directory called "*Lucene*" that contains the source files of the indexing and term search program developed in *section 2.1*.
3. A file called "*memoriaPractica6.pdf*" that includes:
 - a. The development of the practice.
 - b. Answers and reflections on the block 1 of the practice (*Lucene*), including the searched words and frequency/relevance obtained from the application developed.
 - c. Brief explanation of the development of block 2 of the practice (*Weka*) and an analysis of the results obtained in each of the analyses performed with the tool. In other words, it should include the results obtained from *Weka*, but above all, it is necessary to explain the interpretation of those results, what new knowledge we obtain at a glance of the obtained results.
 - d. The methodology used for developing the practice (resources, tools used, work distribution, working hours, etc.), the difficulties found during the realization of the practice.

When extracting the files from the .tar file, you must extract the files and directories indicated in the *practica6_Nip1_Nip2_y_Nip3.tar* directory. It is important to follow the naming conventions and file structure described.

A planning session has been scheduled for this practice. Therefore, the deadline for submission is December 22nd. For submitting the .tar file, you will use the corresponding link available on the course page in the *Anillo Digital Docente (ADD)* of the University of Zaragoza on the Moodle 2 platform (<http://moodle.unizar.es>). In case it is not possible to submit the file through ADD, you will send the tar file to your professor via email.

5.- Correction Procedure and Recommendations

Once the practice is completed and submitted, each group must present it to their practice professor during the last practice session. This will serve as an opportunity for the professor to review the practice, clarify any doubts, and provide recommendations before the exam. During the presentation, the professor will ask questions about the design and implementation decisions made by the workgroup.

- Minimum requirements to meet (10 points)
 - Developed application and analysis conducted in block 1 of the practice (4 points).
 - Analysis of results obtained in block 2 (4 points)
 - Technical report on project realization (2 points).